



Intelligence Artificielle - IA

TP01 - Python - Numpy

Dr. Mohamed Ould Djibril

October 26, 2023

Département d'Informatique et Méthodes Quantitatives
Institut Supérieur de Comptabilité et d'Administration
des Entreprises - ISCAE

mohamed.djibril@gmail.com



1

Sommaire

- Manipulation des vecteurs
- Manipulation des matrices
- Les tableaux multidimensionnels
- Slicing vecteur
- Conversion tableau-vecteur

2

Récapitulative



Instructions

Notes:

- Suivre les instructions ci-dessous étape par étape.
- Executer les codes et corriger les exercices dans un cahier **Jupyter** qui porte le nom : *VotreID_IA_TP01.ipynb*.
- Répondre aux questions et compléter les codes en salle de classe ou à la Maison.
- Envoyer le fichier par mail à : mohamed.djibril@iscae.mr.
- Date du rendu sera communiquée durant les séances du cours.

Numpy



numpy aide à effectuer des calculs numériques en bloc.

- **numpy** est la librairie incontournable pour faire des calculs en Python.
- Ses fonctionnalités utilisent les optimisations processeurs.
- Les principaux objets de **numpy** sont des tableaux.
- Nous y allons progressivement en commençant par les vecteurs simple et les fonctions d'une variable.
- On appelle le module **numpy** et on le renomme de façon raccourcie **np**:

```
import numpy as np
```



Manipulation des vecteurs

Définition d'un vecteur: 1/3

- On crée un vecteur à partir d'une liste :

```
X = np.array([1,2,3,4])
```

- Affichage par `print(X)` :

```
[1 2 3 4]
```

Cela ressemble beaucoup à une liste, mais ce n'est pas une liste usuelle de Python.

- Le type de X est `numpy.ndarray`. Notez qu'il n'y a pas de virgules dans l'affichage du vecteur.

Exercice:

Executer le code suivant:

```
import numpy as np  
  
a = np.array([1, 4, 5, 8], float)  
print(a)  
type(a)
```



Manipulation des vecteurs

Définition: U.N.E.M. 2/3

- Définition: Une suite d'éléments avec `arange()`.
- Même comportement que la fonction classique `range()` avec en plus la possibilité d'utiliser un pas non entier.
- Exemple : `np.arange(1,8,0.5)` renvoie :
[1. 1.5 2. 2.5 3. 3.5 4. 4.5 5. 5.5 6. 6.5 7. 7.5]

Exercice: Liste de comp.

Les listes de compréhension sont de la forme :

[exp for i in range(n)]

Qu'affiche le code suivant?

```
X=np.array([i*i for i in range(5)])
print(X)
```



Manipulation des vecteurs

Définition U.N.E.M. Exercice 3/3:

- Définition d'une division d'intervalle avec `linspace()`.
- C'est l'une des méthodes les plus utilisées.
- Elle s'utilise avec la syntaxe `linspace(a,b,num=n)` pour obtenir une subdivision régulière de $[a, b]$ de n valeurs (donc en $n - 1$ sous-intervalles).

Exercice :

On pose :

```
X = np.linspace(0,1,num=12)
```

- Afficher X
- Quel est le nombre de valeurs de X
- Quelle est la dernière valeur du vecteur ?
- Quelle est le type de X



Manipulation des vecteurs

Les types de données:

- Les éléments d'un tableau numpy doivent être d'un type basique, par exemple entiers ou flottants. (voir la docu.)
- Les éléments d'un tableau numpy doivent être du même type, par exemple des flottants de 64 bits, ou des entiers de 16 bits.
- Connaissant le type d'un tableau et le nombre de ses éléments, vous pouvez estimer facilement la taille du tableau en mémoire.

Reprendre le code ci-dessous ? Quelle est la taille en mémoire de chaque tab. ?

```
from sys import getsizeof as sizeof
# ici numpy detecte automati. le type
x = np.array([0, 1, 2])
print(x.dtype)
# ici numpy detecte automati. le type
x = np.array([0., 1., 2.])
print(x.dtype)
# ici nous spécifions le type,
# sera interprété par numpy comme int64
x = np.array([0., 1., 2.], dtype=int)
print(x.dtype)
# ici nous spécifions que
# le type est entier codé sur 8 bits
x = np.array([0, 1, 2], dtype=np.int8)
print(x.dtype)
```


Numpy

ISCAF



Manipulation des vecteurs

Opérations élémentaires:

Définissons X et Y par :

```
X = np.array([2,3,4])
```

```
Y = np.array([10,11,12,13])
```

- Multiplication par un scalaire.
Exemple : $2*X$ renvoie le vecteur $[2 \ 4 \ 6 \ 8]$.
- Addition d'une constante.
Exemple : $X+1$ renvoie le vecteur $[2 \ 3 \ 4 \ 5]$.
- Somme terme à terme.
 $X+Y$ renvoie $[11 \ 13 \ 15 \ 17]$.
- Produit terme à terme.
 $X*Y$ renvoie $[10 \ 22 \ 36 \ 52]$.

Exercice :

Les données du contrôle continu sont:

$CC = (9 \ 13 \ 10.5 \ 6 \ 1.5 \ 8)$

$CTP = (14 \ 15 \ 15.5 \ 12 \ 10.5 \ 14)$

- Définir les vecteurs CC et CTP .
- Le professeur a décidé d'ajouter 1.0 à tous ces étudiants, recalculer CC .
- Calculer la note finale du contrôle donnée par:
 $CF = (CC + 2 \times CTP)/3$.

Numpy



Manipulation des vecteurs

Opérations :

Définissons un vecteur X par :

```
X = np.array([1,2,3,4])
```

- Carré.

Exemple : $X**2$ renvoie le vecteur
[1 4 9 16].

- Inverse.

Exemple : $1/X$ renvoie le vecteur
[1. 0.5 0.33333333 0.25].

Exercice :

Qu'affiche le code ci-dessous ?

```
import numpy as np
X=np.arange(4, dtype=float)
1/X
X**2
```

Numpy



Manipulation des vecteurs

Opérations :

Définissons un vecteur X par :

```
X = np.array([1,2,3,4])
```

- Somme.
Exemple: `np.sum(X)` renvoie le nombre 10.
- Minimum, maximum.
Exemple : `np.min(X)` renvoie 1 et `np.max(X)` renvoie 4.

Exercice :

Qu'affiche le code ci-dessous ?

```
import numpy as np  
n = 10  
print(np.arange(n).sum())
```



Manipulation des vecteurs

Initialiser un vecteur :

- Avec des zéros.
`np.zeros(6)` renvoie
`[0. 0. 0. 0. 0. 0.]`
- Avec des uns.
`X = np.ones(5)`, alors X vaut
`[1. 1. 1. 1. 1.]`.
- Avec des nombres aléatoires entre 0 et 1.
`np.random.random(5)` renvoie un vecteur dont les coordonnées sont tirées aléatoirement à chaque appel, on obtient par exemple :
`[0.21132407 0.30685886 0.94111979 0.39597993 0.63275735]`

Exercice : Attention aux types

Comparer et commenter les deux codes suivants :

```
tab = np.arange(12)
tab[0] = np.pi
tab[0].dtype, tab[0]
```

```
tab1 = tab.astype(np.float64)
tab1[0] = np.pi
tab1[0].dtype, tab1[0]
```

Numpy



Manipulation des vecteurs

Accéder à un élément :
X = np.linspace(1, 2, num=10)

- Les indices commencent à 0.
- Les indices peuvent être négatifs: ils désignent les éléments comptés à partir de la fin.
- Récupérer une coordonnée. X[0] renvoie le premier élément de X, X[1] renvoie le second élément, etc.
- Tous les éléments. X[:] renvoie tous les éléments de X,
- X[-1] renvoie le dernier élément de X,
- Longueur d'un vecteur. len(X) ou X.size renvoie le nombre d'éléments.
- La taille du tableau ou dimension np.shape(X)



Manipulation des vecteurs

Accès élément : Slicing

X = np.arange(1, 5)

- On peut avoir accès à des morceaux de vecteurs:
`[deb(inclu):fin(exclu)]` ou bien
`[deb(inclu):fin(exclu):increment]`
- Par défaut deb vaut 0, fin vaut -1 et increment vaut 1.
- `X[1:3]` renvoie `array([2, 3])`, l'incrément est de 1.
- `X[-3:-1]` renvoie `array([2, 3])`, l'incrément est de 1.
- `X[2:]` renvoie `array([3, 4])`, on prend jusqu'au dernier.
- `X[1::2]` renvoie `array([2, 4])`, l'incrément vaut 2.
- `X[::-1]` renvoie `array([4, 3, 2, 1])`, lecture à l'envers.



Manipulation des vecteurs

Aliasing d'un tableau:

Soient X et Y deux ndarrays :

- Aliasing par défaut: $Y=X$ ne crée pas un nouveau tableau mais un alias.
- De même $Y=X[:]$ ne crée pas un nouveau tableau mais un alias.
- La copie explicite : $Y[:]=X[:]$
- ou : $Y = X.copy()$

Exercice :

Qu'affiche le code ci-dessous ?

```
X=np.zeros(2)
Y=X
Z=np.zeros(2)
Z[:]=X[:]  
Y[0]=1  
Z[0]=2  
print("X=",X,"Y=",Y,"Z=",Z)
```

Numpy



Manipulation des vecteurs

Exercice Manipulation des vecteurs

Créer un vecteur X : `X=np.random.rand(100)`

- 1 Mettre dans la variable n le nombre d'éléments du vecteur X.
- 2 Afficher à l'écran la valeur du 3eme élément de X.
- 3 Construire un vecteur qui contient les éléments `X[8]`, `X[9]`, ... `X[20]`.
- 4 Construire un vecteur qui contient les éléments `X[8]`, `X[10]`, `X[12]`, ... `X[20]`.
- 5 Echanger le 5eme et le 7eme élément (deux façons : une très informatique, l'autre en utilisant le slicing).
- 6 Créer un vecteur Y qui contient tous les éléments de X lus dans le sens inverse.



Manipulation des vecteurs

le Slicing

on crée un numpy.ndarray de dimension 1 de taille 10

```
X = np.arange(10)
```

- on prend un élément sur 2 en partant du début de la liste
- on modifie les éléments du sous-tableau obtenu
- le tableau initial est modifié

```
X = np.arange(10) # [0 1 2 3 4 5 6 7 8 9]
print(X[::2])     # [0 2 4 6 8]
X[::2] = 100
print(X)          # [100, 1, 100, 3, 100, 5, 100, 7, 100, 9]
```

Numpy



Manipulation des vecteurs

Parcourir

Parcourir tous les éléments.

- Première méthode :

```
for x in X:  
    print(x)
```

- Deuxième méthode :

```
for i in range(len(X)):  
    print(i,X[i])
```

Attention :

Eviter d'utiliser les boucles dans ce cours.



Manipulation des vecteurs

Notes : U.N.E.M. les et le slicing

Si l'on doit effectuer des boucles sur un très grand nombre de points, le temps de restitution risque de devenir très long.

Dans ce cas on préférera faire du "slicing" c'est à dire faire une implémentation vectorisé quand cela est possible.

Executer et commenter le code:

```

nx=1000000
f1=np.zeros(nx)
f2=np.zeros(nx)
g=np.ones(nx)

# boucle
t0=time.time()
for i in np.arange(1,nx-1):
    f1[i]=g[i+1]+g[i]+g[i-1]
t1=time.time()-t0

# slicing
t0=time.time()
f2[1:-1]=g[2:]+g[1:-1]+g[0:-2]
t2=time.time()-t0
print(t1,t2,t1/t2)

```

Numpy



Manipulation des vecteurs

Application d'une fonction sur un vecteur :

Avec Numpy, on peut appliquer une fonction directement sur chaque coordonnée d'un vecteur.

Prenons l'exemple de : `X = np.array([0,1,2,3,4,5])`

- Racine carré.

Exemple : `np.sqrt(X)` renvoie le vecteur

`[0. 1. 1.41421356 1.73205081 2. 2.23606798].`

- Puissance.

Exemple : `X**2` renvoie le vecteur `[0 1 4 9 16 25].`



Manipulation des vecteurs

Application d'une fonction sur un vecteur :

Prendre en exemple de : `X = np.array([0,1,2,3,4,5])`

- Les fonctions mathématiques essentielles sont implémentées ainsi que la constante π : `np.pi`.
- `np.cos(X)` (calcul des cosinus, unité d'angle le radian)
- `np.cos(2*np.pi/360*X)` (unité d'angle le degré).

Note importante :

Il est important d'utiliser `np.sqrt(X)` plutôt qu'une boucle du type `for x in X: np.sqrt(x)` car d'une part le code est plus lisible, mais surtout les calculs sont optimisés.

En effet, Numpy profite de la présence de plusieurs processeurs (ou cœurs) dans la machine et effectue les calculs en parallèle. Les calculs sont beaucoup plus rapides.

Numpy



Manipulation des vecteurs

Applique une fonction sur un vecteur :

Voici les principales fonctions. Il en existe plein d'autres !

<code>x ** a</code>	
<code>sqrt(x)</code>	
<code>exp(x)</code>	
<code>log(x)</code>	
<code>log10(x)</code>	
<code>cos(x), sin(x), tan(x)</code>	
<code>arccos(x), arcsin(x), arctan(x)</code>	
<code>cosh(x), sinh(x), tanh(x)</code>	

Numpy



Manipulation des vecteurs

Note importante :

Attention ! Numpy se substitue au module `math` qu'il ne faut pas utiliser. (Par exemple la commande `math.cos(X)` renverrait une erreur.)

Numpy



Manipulation des matrices

Une matrice est un tableau de n lignes et p colonnes, contenant des nombres réels.

$$\begin{matrix} \updownarrow n \text{ lignes} & \begin{pmatrix} a_{11} & a_{12} & \dots & a_{1p} \\ a_{21} & a_{22} & \dots & a_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \dots & a_{np} \end{pmatrix} & \leftarrow p \text{ colonnes} \end{matrix}$$

Si $n = p$, on parle de **matrice carrée**. Voici une matrice 3×3 :

$$\begin{pmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{pmatrix}$$

Numpy



Manipulation des matrices

Addition et multiplication par un scalaire

On peut additionner deux matrices de même taille.
L'addition se fait terme à terme. Exemple :

$$\begin{pmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{pmatrix} + \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix} = \begin{pmatrix} 1 & 3 & 3 \\ 5 & 1 & 7 \\ 7 & 9 & 9 \end{pmatrix}$$

- **Multiplication par un scalaire.** On peut multiplier une matrice par un nombre réel. Exemple :

$$3 \cdot \begin{pmatrix} 1 & 2 & 3 & 4 \\ 8 & 7 & 5 & 6 \end{pmatrix} = \begin{pmatrix} 3 & 6 & 9 & 12 \\ 24 & 21 & 15 & 18 \end{pmatrix}$$

Numpy



Manipulation des matrices

Règles de multiplication matricielle

- **Multiplication matricielle.** On peut multiplier deux matrices.
- Condition : nombre de lignes de la première matrice égal au nombre de colonnes de la deuxième matrice.
- La multiplication **n'est pas** effectuée terme à terme.

$$\begin{pmatrix} -2 & -1 & 0 \\ -1 & 1 & 1 \\ 0 & 1 & 2 \end{pmatrix} \times \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix} = \begin{pmatrix} -6 & -9 & -12 \\ 10 & 11 & 12 \\ 18 & 21 & 24 \end{pmatrix}$$



Manipulation des matrices

Définition d'une matrice numpy

Une matrice à deux dimensions ou une **matrice** est un tableau à deux entrées.

- Un tableau se définit comme une suite de lignes
`A = np.array([[1, 2, 3], [4, 5, 6]])`

- s'affiche ainsi :

```
[[1 2 3]
 [4 5 6]]
```

- C'est un tableau à deux lignes et trois colonnes qui correspond à la matrice :

$$\begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix}.$$



Manipulation des matrices

`np.zeros((n,p))` renvoie un tableau de n lignes et p colonnes rempli de 0.

- La fonction `np.ones()` fonctionne sur le même principe.
- `np.eye(2)` # *eye* -> *matrice identité*

Taille d'une matrice

- On récupère la taille du tableau par la fonction `shape()`.
- Exemple: soit `A = np.array([[1, 2, 3], [4, 5, 6]])`
`np.shape(A)` renvoie ici (2,3), pour 2 lignes et 3 colonnes.

Numpy



Manipulation des matrices

E.

- Définir les matrices suivantes.

$$A = \begin{pmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{pmatrix} \quad B = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix} \quad C = \begin{pmatrix} -2 & -1 & 0 & 0 \\ -1 & 1 & 1 & 0 \\ 0 & 1 & 2 & 0 \end{pmatrix}$$

- Afficher à l'écran la taille (dimension) de chaque matrice.
- Afficher à l'écran le nombre d'éléments de chaque matrice, indication : utiliser `size`.

Numpy



Manipulation des matrices

Accès aux éléments d'une matrice:

Accès aux éléments par des instructions du type $A[i, j]$

- où i est le numéro de ligne.
- et j celui de la colonne.

Exercice:

Soit le tableau `arr` défini ci-dessous:

```
arr = np.array([[1,2,3,4,5], [6,7,8,9,10]])
```

alors compléter en commentaires le code suivant:

```
print(arr[0, 1]) # affiche le second élém. de la prem. ligne.  
print(arr[1, 4]) # affiche _____  
print(arr[1, -1]) # affiche _____  
print(arr[:, :]) # affiche _____
```

Numpy



Manipulation des matrices

Exemples d'une matrice: Slicing

Dans l'exemple ci-dessus:

- on récupère les colonnes 2 et 3 de toutes les lignes.
- Le : à gauche de la virgule, indique que l'on désire récupérer les 5 lignes.
- Le 2:4 à droite de la virgule, indique que l'on désire récupérer les colonnes 2 à 4 (exclus) donc les colonnes 2 et 3.

```
matrice = np.array([[0,1,2,3], [4,5,6,7], [8,9,10, 11],  
                  [12, 13, 14, 15], [16,17,18,19]])  
matrice[:,2:4]
```

Numpy



Manipulation des matrices

Exercice 1 : slicing et affectation

Reprendre le code suivant et expliquer les résultats

```
A = np.eye(5,5)
print(A)
A[:3,:4]=4
A[:,3]=5
A[:,3]=6
print(A)
```




Manipulation des matrices

Opérations élémentaires: L'addition

À l'aide de l'opérateur (+) entre un ndarray et un scalaire, cela va ajouter la valeur du scalaire à tous les éléments du ndarray.

- La soustraction, la * et la / fonctionnent de la même manière.

Exemple:

```
vecteur = np.array([0,1,2])  
vecteur + 1  
renvoie : array([1, 2, 3])  
matrice = np.array([[1,2,3], [3,4,5], [5,6,7]])  
matrice + 1  
renvoie : [[2, 3, 4], [4, 5, 6], [6, 7, 8]]
```

Numpy



Manipulation des matrices

Vocabulaire:

- **Elementwise** properties, en français, les propriétés **éléments par éléments**, sont les fonctionnalités de Numpy qui permettent de procéder à des modifications sur les ndarray élément par élément.

Numpy



Manipulation des matrices

Opérations élémentaires: L'addition

On peut additionner la ligne d'une matrice et un vecteur si leurs dimensions correspondent.

- Dans l'exemple suivant, on va additionner le vecteur à la première ligne de notre matrice.

Exemple:

```
vecteur = np.array([0,1,2])  
matrice = np.array([[1,2,3], [3,4,5], [5,6,7]])  
  
matrice[0] + vecteur  
renvoie : array([1, 3, 5])
```

Numpy



Manipulation des matrices

Opérations élémentaires: L'addition

• Additionner le vecteur à toutes les lignes de la matrice.

- Ci-dessous, on va addit. le vecteur à ttes les lignes de la matrice.

Exemple:

```
vecteur = np.array([0,1,2])
matrice = np.array([[1,2,3], [3,4,5], [5,6,7]])
matrice + vecteur

#renvoie
array([[1, 3, 5],
       [3, 5, 7],
       [5, 7, 9]])
```

Numpy



Manipulation des matrices

Application d'une fonction à une matrice:

- Les fonctions s'appliquent élément par élément.
- Par exemple `np.sqrt(A)` renvoie un tableau ayant la même forme, chaque élément étant la racine carrée de l'élément initial.



Manipulation des matrices

A. Appliquer une fonction à une matrice:

- La somme de ses éléments :
`np.sum(M)` # *Peut aussi s'écrire M.sum()*
- Sa plus petite / plus grande valeur :
`np.min(M)` # *Peut aussi s'écrire M.min()*
- La moyenne de ses éléments :
`np.mean(M)` # *Peut aussi s'écrire M.mean()*
- Il est à noter que pour toutes ces opérations, deux syntaxes co-existent : `print(np.min(M))` `print(M.min())`

Numpy



Manipulation des matrices

A.U.N.E.M. une fonction à une matrice:

- On peut également effectuer ces opérations ligne par ligne, ou colonne par colonne, comme :
On somme sur les lignes (dimension numéro 0)
Donc on obtient un résultat par colonne
`M.sum(axis=0)`
- On peut créer des masques binaires, ce qui permet de compter simplement le nombre de valeurs d'un tableau vérifiant une condition :
`np.sum(M > 5)`

Numpy



Manipulation des matrices

Supposons qu'on ait stocké dans le tableau suivant les notes reçues par 2 étudiants à 3 examens :

```
notes = np.array(  
    [[10, 12],  
     [15, 16],  
     [18, 12]]  
)
```

- 1- Calculez la moyenne de chacun des deux étudiants.
- 2- Calculez le nombre de notes supérieures à 12 contenues dans ce tableau

Numpy



Manipulation des matrices

Exemple de code usuel :

```
M1 = np.array([ [1.,1.,1.],  
               [1.,3.,1.],  
               [1.,1.,1.]])  
M2 = np.array([ [1.,1.,1.],  
               [1.,2.,1.],  
               [1.,1.,1.]])  
MResultAdd = np.add(M1,M2)  
MResultSous = np.subtract(M1,M2)  
MResultMulti = M1.dot(M2)
```

Le principe des matrices:

Il y a 2 types de produit: '

- Le produit élément par élément:
 - La fonction **np.multiply()** de NumPy effectue le produit de 2 matrices élément par élément.
 - Utiliser aussi l'opérateur ***** pour ce type de produit.
- Le produit matriciel (ligne-colonne):
 - La fonction **np.matmul()** prend deux matrices en entrée et renvoie le produit si la multiplication matricielle entre les matrices en entrée est possible.
 - L'opérateur binaire **@** est aussi utilisé pour la multiplication de matrices depuis Python 3.5.
 - La fonction **dot()** vu précédemment réalise le produit scalaire de deux tableaux. Le résultat est le même que la fonction **matmul()** pour les tableaux unidimensionnels et bidimensionnels.

Numpy



Manipulation des matrices

deux type de produits matriciels

Reprendre et expliquer la différence entre les deux résultats.

```
import numpy as np

a1 = np.array([[1, 2], [3, 4]])
a2 = np.array([[5, 6], [7, 8]])
a3 = np.multiply(a1, a2)
print(a3)

m3 = np.matmul(a1, a2)
print(m3)
```

Numpy



tableaux multidimensionnels

Un tableau à une entrée est un vecteur

- Un tableau à deux entrées est une matrice
- Un tableau peut avoir d entrées avec $d \geq 2$ on parle alors des **tableaux multidimensionnels**.

Définition:

- Les tableaux multidimensionnels peuvent être créés à partir d'une liste de listes, par exemple pour $d = 2$:

```
A = np.array([[0, 1], [2, 3], [4, 5]])  
et A.shape renvoie (3, 2)
```

Numpy



tableaux multidimensionnels

accès à un élément: exemple pour ($d = 3$)

```
A = np.array([1, 2, 3], [4, 5, 6], [[7, 8, 9], [10, 11, 12]])
```

A.shape renvoie (2,2,3) et `print(A[0, 1, 2])` affiche 6

Processus de lecture:

Le premier nombre représente la première dimension, qui contient deux tableaux: `[[1, 2, 3], [4, 5, 6]]` et: `[[7, 8, 9], [10, 11, 12]]`

L'indice 0 pour la première dimension est donc le premier tableau:

```
[[1, 2, 3], [4, 5, 6]]
```

Le deuxième nombre représente la deuxième dimension, qui contient aussi deux tableaux: `[1, 2, 3]` et: `[4, 5, 6]`

L'indice 1 pour la deuxième dimension est donc le deuxième tableau :

```
[4, 5, 6]
```

Le troisième nombre représente la troisième dimension, qui contient trois valeurs: 4 5 6

L'indice 2 pour la troisième dimension est donc la valeur : 6

Numpy



tableaux multidimensionnels

Créer une image *img* de 2x2 pixels, avec 3 canaux de couleurs (rouge, bleu, vert) dans chaque pixel

```
x = np.array(  
    [  
        [ [1,2,3], [4,5,6], ],  
        [ [7,8,9], [10,11,12]]  
    ]  
)  
print(x)  
print(x.shape)
```

Quelle est la valeur du canal bleu du pixel (1,1)?



Conversion tableau-vecteur

U.N.E.M. : Matrice vers vecteur

- Il est souvent utile de passer d'un tableau à un vecteur et réciproquement.
- On obtient tous les éléments d'un tableau regroupés dans un vecteur par la commande d'aplatissement `flatten()`.
- Exemple : si `A = np.array([[1, 2, 3], [4, 5, 6]])` alors la commande `X = A.flatten()` renvoie le vecteur `X` :
`[1 2 3 4 5 6]`
- La même opération peut être réalisée avec :
`X = A.reshape(-1)`



Conversion tableau-vecteur

U.N.E.M. matrice

- Pour transformer un vecteur en une matrice on utilise la fonction python **reshape()** qui prend en entrée la nouvelle taille désirée.
- Exemple : soit le vecteur $X : [1 \ 2 \ 3 \ 4 \ 5 \ 6]$ alors :
 $X.reshape((2,3))$ redonne exactement A.
- Par contre $X.reshape((3,2))$ renvoie un tableau à 3 lignes et 2 colonnes :

```
[[1 2]
 [3 4]
 [5 6]]
```


Numpy



Conversion tableau-vecteur

Noter bien:

- **flatten** renvoie toujours une copie du tableau.
- **reshape** est plus flexible car vous pouvez remodeler n'importe quoi. Il renvoie une vue et n'effectue une copie que lorsque cela est nécessaire.

Numpy



former un tableau 1D en un tableau 1D "colonne"

Verone en colonne

ons transformer un tableau de forme $(N,)$ en un tableau de forme $(N,1)$.

- Nous le transformons le tableau y de forme $(5,)$ en colonnes $(5,1)$ avec l'attribut `c_` (notez que ce n'est pas une fonction !)

Exercice:

Executer et commenter le code :

```
y = np.arange(5)
print(y)
print(y.shape)
```

```
coly = np.c_[y]
print(coly)
```



1 minute pour m'autoévaluer !

spécifiques:

- 1 Je sais définir un vecteur à une ou à plusieurs dimensions avec plus d'une méthode.
- 2 Je sais afficher sa dimension et connaître le nombre de ses éléments.
- 3 Je peux assigner un type aux éléments d'un tableau. je sais aussi le changer.
- 4 Je peux accéder à un ou plusieurs éléments de mon tableau.
- 5 Je me sens bien avec le slicing !!
- 6 Je maîtrise l'utilisation des opérations élémentaires un tableau et je connais les conditions nécessaires et suffisantes.
- 7 Je maîtrise l'application des fonctions usuelles sur un tableau.

Récapitulative



Suivant

Nous allons étudier :

Vous en savez maintenant suffisamment sur numpy pour gérer la plupart des opérations rencontrées dans les projets de deep learning. Dans le TP suivant:

- Nous allons attaquer la matplotlib
- Tracer les graphes des fonctions